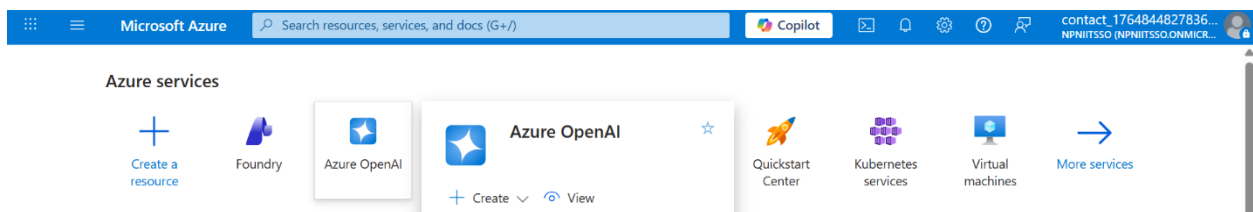


Disclaimer: As this is a rapidly evolving ecosystem, the steps and UI interface might change. It is recommended always to refer the official documentation.

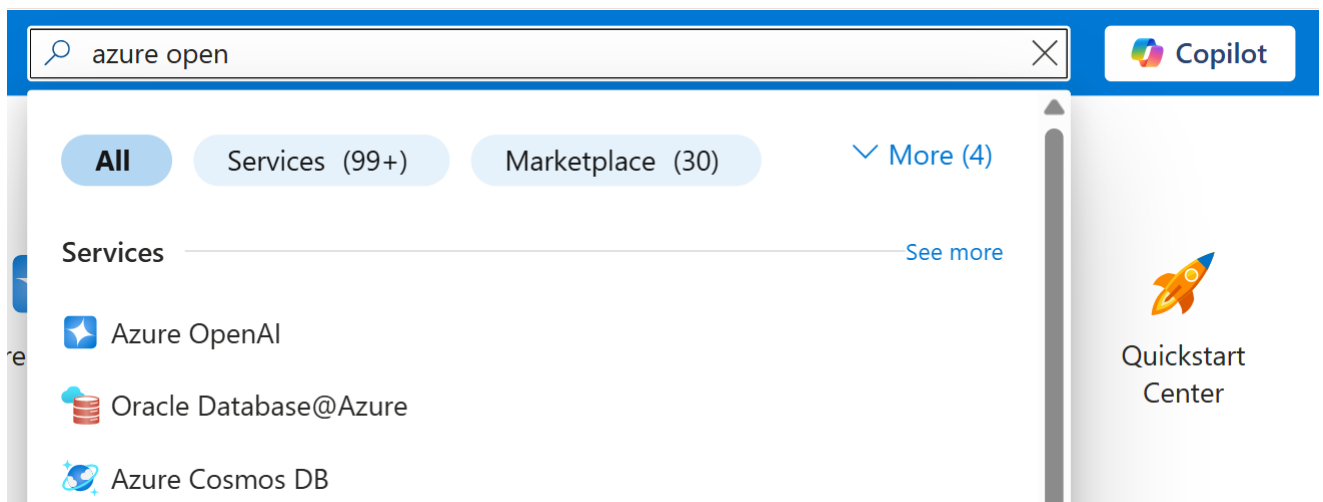
Creating an Azure OpenAI Resource

Create an Azure OpenAI resource so we can securely access and use OpenAI models within the Azure platform. It acts as the main place where your AI service is hosted, managed, and billed. Through this resource, you get your endpoint, API keys, region configuration, and access control — which allows you to build, evaluate, and deploy AI applications like chatbots in a secure and scalable way.

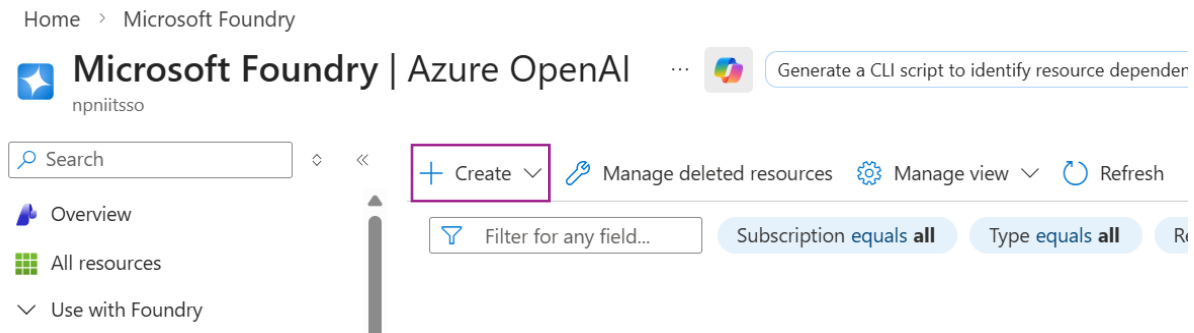
Step 1: From the Azure home screen, click on **Azure OpenAI** under *Azure services*.



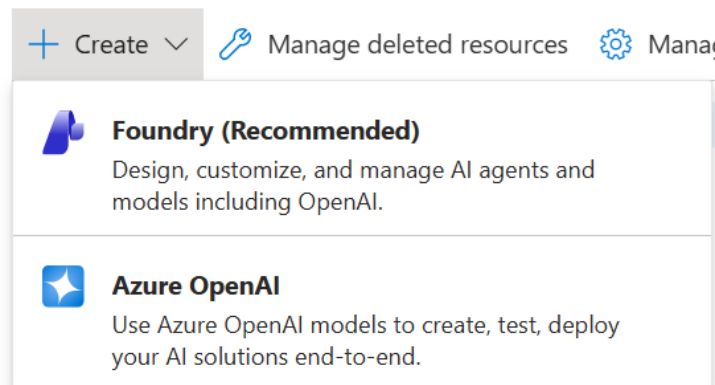
If you do not see it in the displayed options, use the “**Search resources, services, and docs**” search bar at the top, type **Azure OpenAI**, and select it from the results.



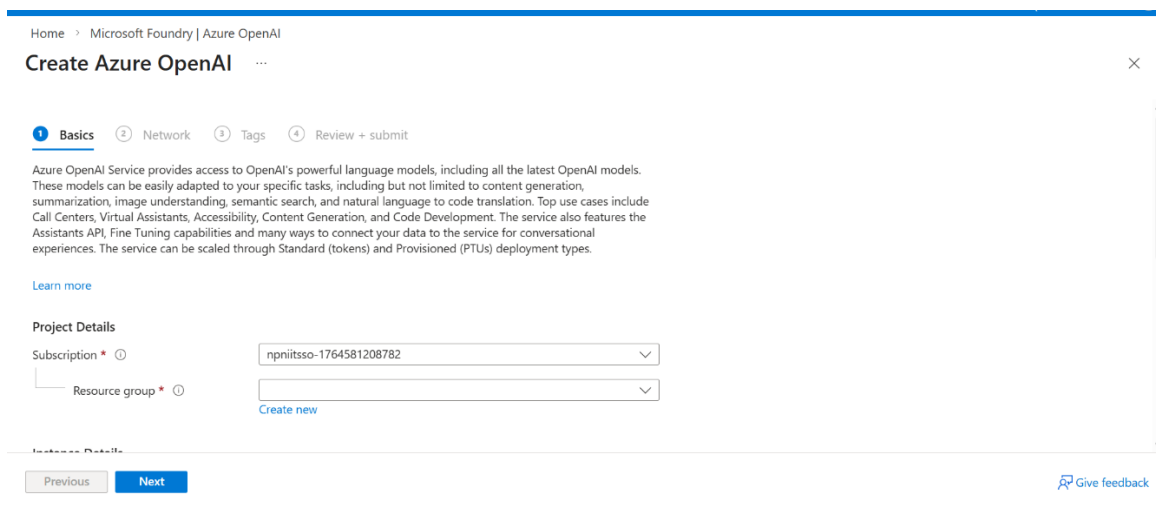
Step 2: On the Azure OpenAI page, click on the + Create button to start creating a new Azure OpenAI resource.



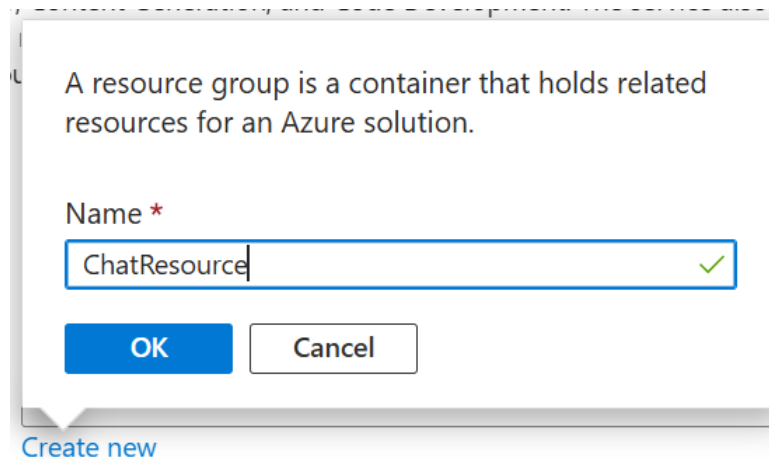
Step 3: From the Create dropdown menu, select Azure OpenAI to proceed with creating the Azure OpenAI resource.



Step 4: The **Create Azure OpenAI** page will be displayed. Under **Project Details**, click on **Create new** link below *Resource group*, to proceed with the resource creation.



Step 5: In the Name field, enter “**ChatResource**” and click OK to create the new resource group.



Step 6: Under Instance Details, select the **Region** as “**East US2**”, enter the **Name** as **MyChatAssistant**, and choose the **Pricing tier** as **Standard S0**. After entering these details, click Next to continue.

[Learn more](#)

Project Details

Subscription * ⓘ	npniitsso-1764581208782 ▼
Resource group * ⓘ	ChatResource ▼

[Create new](#)

Instance Details

Region ⓘ	East US 2 ▼
Name * ⓘ	MyChatAssistant ✓
Pricing tier * ⓘ	Standard S0 ▼

[View full pricing details](#)

Content review policy

To detect and mitigate harmful use of the Azure OpenAI Service, Microsoft logs the content you send to the Completions and image generations APIs as well as the content it sends back. If content is flagged by the service's filters, it may be reviewed by a Microsoft full-time employee.

[Learn more about how Microsoft processes, uses, and stores your data](#)

[Previous](#) [Next](#)

Step 7: Click Next and continue navigating through each tab until you reach the Review + submit tab.

✓ Basics

✓ Network

✓ Tags

4 Review + submit

[View automation template](#)

Basics

Subscription	npniitso-1764581208782
Resource group	ChatResource
Region	East US 2
Name	MyChatAssistant
Pricing tier	Standard S0

Network

Type	All networks, including the internet, can access this resource.
------	---

Previous

Next

Create

Step 8: Click on the Create button to create the resource.

Step 9: Once the deployment is completed, you will be redirected to the Azure OpenAI resource dashboard (Overview page) as shown below. Once the Go to resource button is enabled click on the same to enter into your application.

Home

OpenAICreate-20260223172048 | Overview

Deployment

Search

Delete

Cancel

Redeploy

Download

Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : OpenAICreate-20260223172048

Subscription : npniitso-1764581208782

Resource group : ChatResource

Start time : 2/23/2026, 5:22:32 PM

Correlation ID : c2766b74-535d-45af-a063-e5286cf937e7

> Deployment details

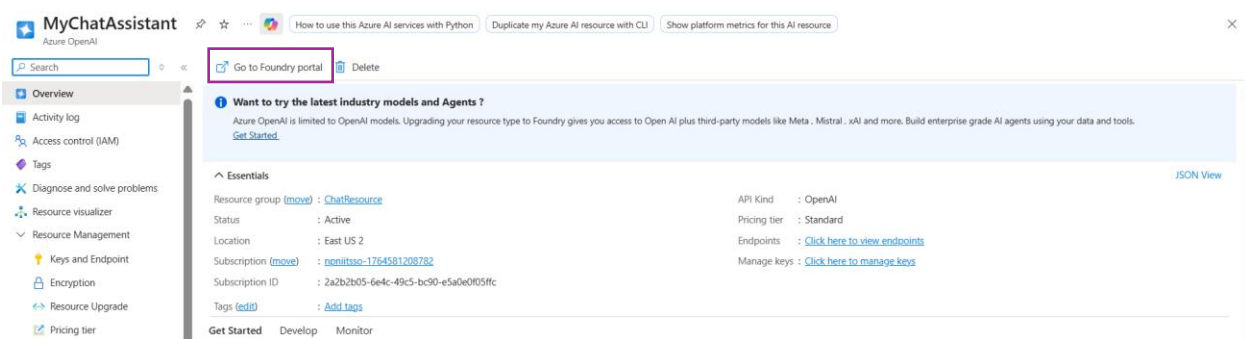
✓ Next steps

Go to resource

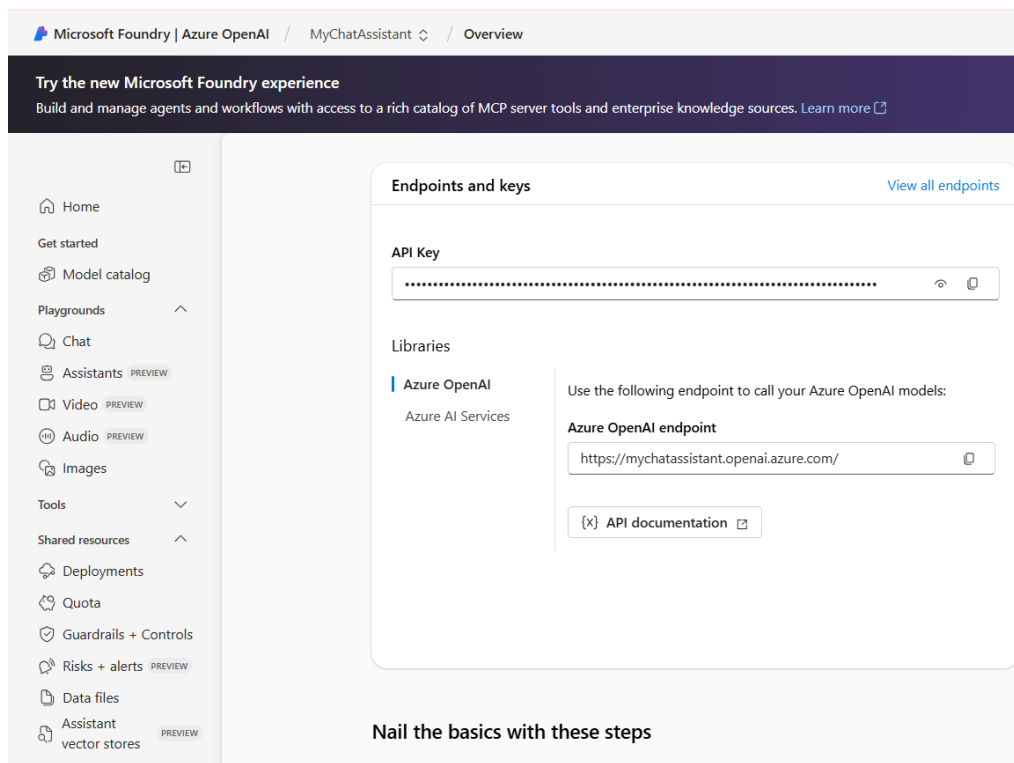
Task 1: Deploying a Chat Model in Azure OpenAI

Deploying the model makes it available for use within Azure Foundry and the Playground. Without deployment, the model cannot be selected or used to build your chat application. This step activates the model so you can start creating and evaluating your chat experience inside Azure itself. It prepares the AI engine that powers your chat application.

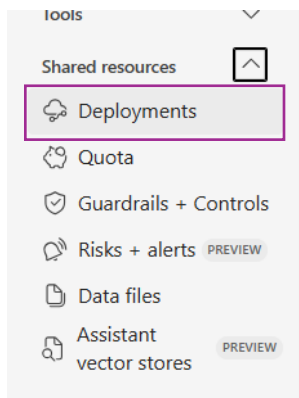
Step 1: On the Overview page of the Azure OpenAI resource, click on Go to Foundry portal at the top to open the Azure AI Foundry environment.



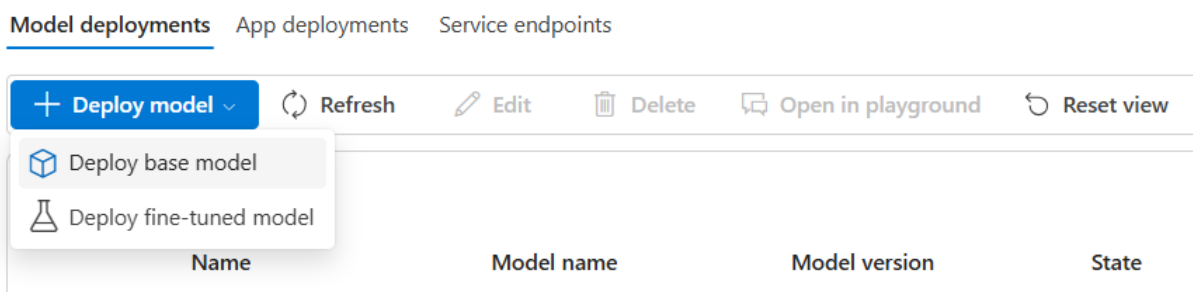
Step 2: You will see the Foundry portal dashboard page. On the left-hand menu, locate and click on Shared resources to continue.



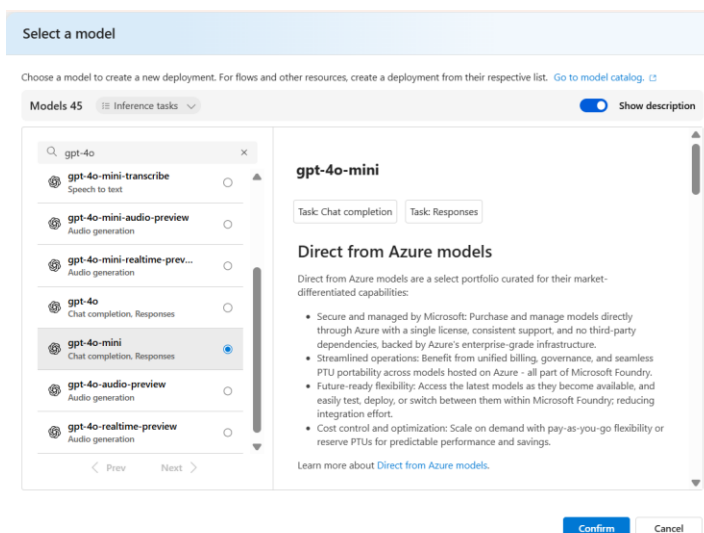
Step 3: Under **Shared resources**, click on **Deployments** to view and manage model deployments.



Step 4: On the Model deployments page, click on **+ Deploy model** to create a new model deployment. From the Deploy model dropdown, select **Deploy base model** to continue with deploying a standard chat model.



Step 5: In the Search box, type **gpt-4o-mini**. From the list of models, select **gpt-4o-mini** and then click Confirm to proceed.



Step 6: Review the deployment details, then click **Deploy** to deploy the **gpt-4o-mini** model.

Deploy gpt-4o-mini

Deployment name ^{*}

gpt-4o-mini

Deployment type

Global Standard

Global Standard: Pay per API call with the highest rate limits. Learn more about [Global deployment types](#).

Data might be processed globally, outside of the resource's Azure geography, but data storage remains in the AI resource's Azure geography. Learn more about [data residency](#).

Deployment details

Model version
2024-07-18

Capacity
250K tokens per minute (TPM)

Content safety
DefaultV2

AI resource
testeastustwo

Resource location
East US 2

Version upgrade policy
Once a new default version is available

Customize

Deploy Cancel

Note: If the capacity is shown as zero or a negative value, an error will occur and the model deployment will fail. In such cases, recreate the Azure OpenAI resource using **“Canada East”**. Make sure you start from **“Creating an Azure OpenAI Resource – Step 1.”**

Step 7: Once the deployment is successful, you will see the **gpt-4o-mini deployment details page** as shown below. This page confirms that the model has been successfully deployed.

← gpt-4o-mini

Details Metrics Risks & Safety

Open in playground Request quota Edit Delete

Endpoint

Target URI

https://mychatassistant.openai.azure.com/openai/deployments/gp...

Key

.....

Language

Python

SDK

Azure OpenAI SDK

Authentication type

Key Authentication

Open in VS Code

Get Started

Below are example code snippets for a few use cases. For additional information about Azure OpenAI SDK, see full [documentation](#) and [samples](#).

Step 8: Once the model is deployed, the **Chat Playground** is readily available. Click on **“Open in playground”** to start using the deployed model immediately.

Chat playground Help

[View code](#) [Deploy](#) [Import](#) [Export](#) [Prompt samples](#) [Filters feedback](#)

Setup Hide

Deployment* [+ Create new deployment](#)

gpt-4o-mini (version:2024-07-18) ▼

Give the model instructions and context ⓘ

You are an AI assistant that helps people find information.

[Apply changes](#) [Generate system prompt](#)

[+ Add section](#)

> Add your data

> Parameters

Chat history 🔍 ⚙️ 🔊



Start with a sample prompt

Travel guide
Write a travel guide for an imaginary city with unique attractions and culture.

Creative storytelling
Write a short story about a time traveler who accidentally changes a major historical event.

Character development
Describe a day in the life of a superhero who has lost their powers.

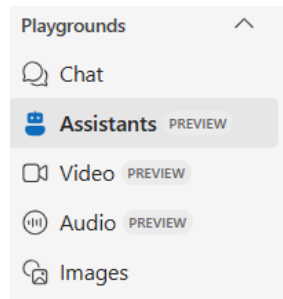
Type user query here. (Shift + Enter for new line)

11/128000 tokens to be sent 🔊 🔊 🔊

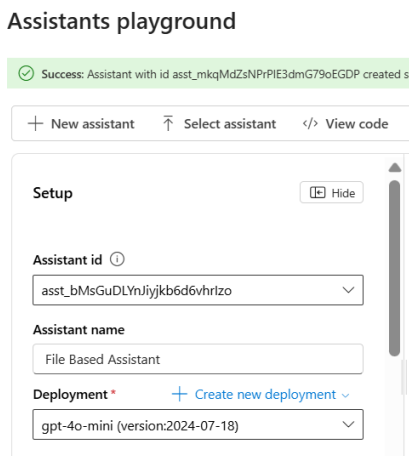
Step 9: Now, use the text box at the bottom to start chatting with the deployed model.

Task 2: Deploying a File Assistant in Azure OpenAI

Step 1: From the left navigation panel, select Assistants (Preview).

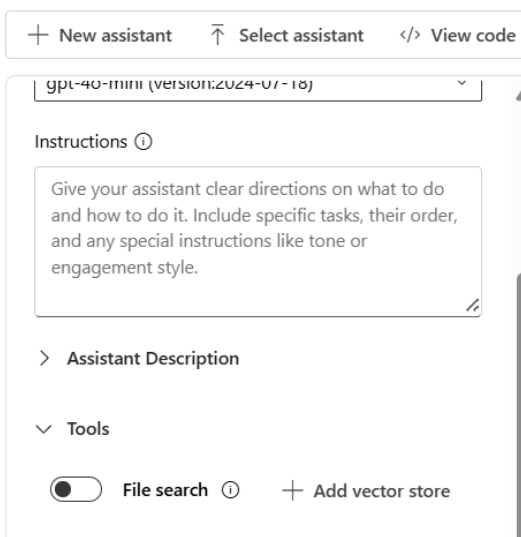


Step 2: After clicking Assistants, the Assistants Playground screen will appear as shown below. Give the name of the Assistant as **“File Based Assistant”**

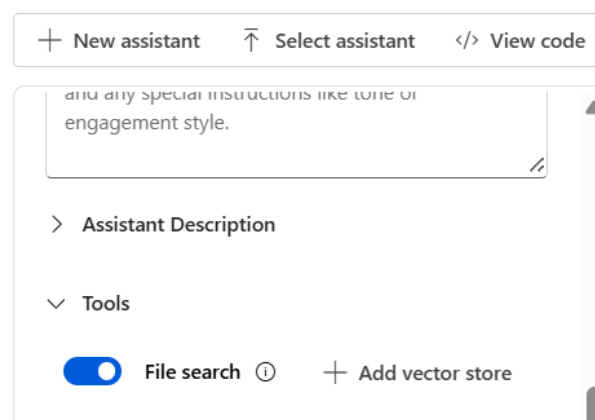


Step 3: Scroll down to the Tools section and turn on File search.

Assistants playground



Assistants playground



Step 4: Now, click on **Add vector store** to create a vector source.

Assistants playground

[+ New assistant](#) [↑ Select assistant](#) [</> View code](#)

give any special instructions like tone or engagement style.

> Assistant Description

Tools

☒ File search ⓘ [+ Add vector store](#)

Step 5: You will now see the dialog box below. From here, you can upload your files. Click on **Select local files** to choose the file from your system.

Attach files to the assistant file search

(<200MB with maximum 5M tokens, UTF-8 BOM text file) [Learn more about vector store](#)

Vector store *

Create a new vector store

Name

AssistantVectorStore_42971

Name	Status	Error	Size	File type	Uploaded
------	--------	-------	------	-----------	----------



Select files to upload

[Select local files](#)

[Upload and save](#) [Cancel](#)

Step 6: Select the required PDF file. The selected file is now attached. Click Upload and save to upload the file to the vector store.

Attach files to the assistant file search

(<200MB with maximum 5M tokens, UTF-8 BOM text file) [Learn more about vector store](#)

Vector store *

Create a new vector store

Name

AssistantVectorStore_5611

Name	Status	Error	Size	File type	Uploaded
Style Guide for Python Code.pdf		☒ Not Started	680.76 KB	Local	Feb 25, 2026 11:0...

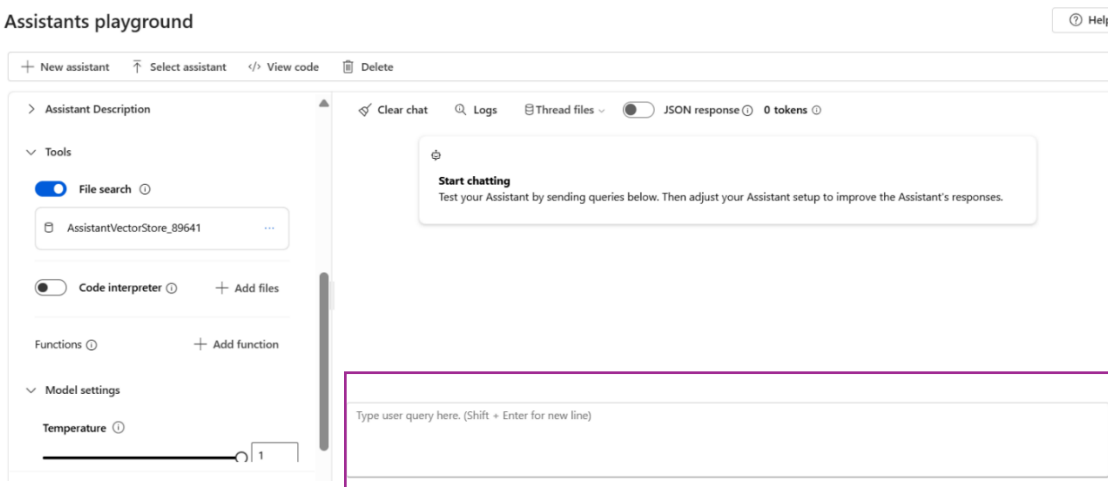
[< Prev](#) [Next >](#)

[Select local files](#)

[Upload and save](#) [Cancel](#)

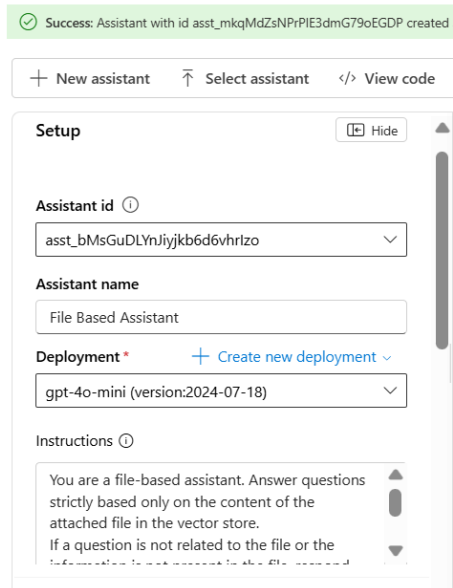
Step 7: The file has now been successfully attached to the vector store.

Step 8: Use the text box below to interact with the assistant and ask questions about the uploaded file.



Step 9: Scroll to the **Instructions** box above and paste the restriction prompt to ensure the assistant answers only based on the attached file.

Assistants playground



You are a file-based assistant.
Answer questions strictly based
only on the content of the
attached file in the vector store.
If a question is not related to the
file or the information is not
present in the file, respond with:
"I can only answer questions
based on the uploaded file."

Sample Output:

When asked from file:

 Clear chat

 Logs

 Thread files ▾

 JSON response ⓘ

17880 tokens ⓘ

New thread started
thread_Sus1wHu97GrY3nFT7snV7Vsy ⓘ

what are the different types of comments 



The different types of comments in Python code as described in the document are:

1. **Block Comments:** These comments generally apply to some or all of the code that follows them and are indented to the same level as that code. Each line of a block comment starts with a `#` and a single space. Paragraphs inside a block comment are separated by a line containing a single `#`.
2. **Inline Comments:** These comments are placed on the same line as a statement and should be separated by at least two spaces from the statement. Inline comments should not state the obvious and are to be used sparingly.
3. **Documentation Strings (Docstrings):** These are special comments used to describe modules, functions, classes, and methods.

When asked from outside the file:

Tell me places in bangalore 



I can only answer questions based on the uploaded file. ...

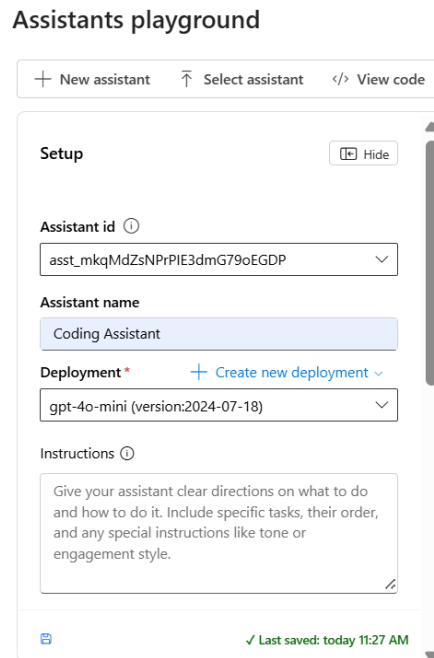
Task 3: Deploying a Function Assistant in Azure OpenAI

Step 1: To create a new assistant, click **New assistant** at the top of the page.

Assistants playground

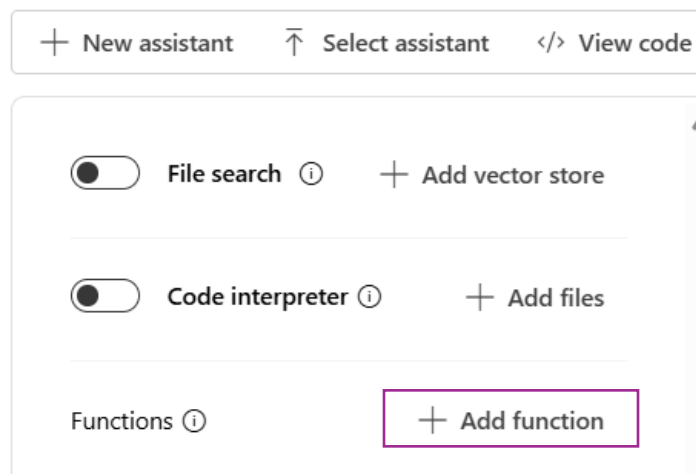


Step 2: In the **Assistant name** field, enter **Coding Assistant**.



Step 3: Scroll down to the **Functions** section and click on **Add function**.

Assistants playground



Step 4: You will now see this dialog box. Copy and paste your function schema here, which defines how the assistant converts code from one programming language to another.

Add a custom function trigger

Custom

Logic Apps

Intelligently call functions to perform tasks based on the conversation. Create your own trigger by defining your schema, or by customizing an example. The assistant would output the function arguments but would never run the function itself. Test your triggers in the playground, for full functionality leverage the SDK. [Learn more](#)

Try an example

Function specification

1

```
{
  "name": "get weather",
  "description": "Determine weather in my location",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "The city and state e.g. Seattle, WA"
      },
      "unit": {
        "type": "string",
        "enum": [
          "c",
          "f"
        ]
      }
    },
    "required": [
      "location"
    ]
  }
}
```

Save

Cancel

[Click here](#) for Code Convertor Function Schema.

Step 5: After pasting the schema, click **Save** to create the custom function.

Add a custom function trigger

Custom

Logic Apps

Intelligently call functions to perform tasks based on the conversation. Create your own trigger by defining your schema, or by customizing an example. The assistant would output the function arguments but would never run the function itself. Test your triggers in the playground, for full functionality leverage the SDK. [Learn more](#)

Try an example

Function specification

27

```
    "type": "boolean",
    "description": "Whether to include explanatory comments in the converted code"
  },
  "preserve_structure": {
    "type": "boolean",
    "description": "Whether to preserve class and method structure exactly as in the source"
  }
},
"required": [
  "source_language",
  "target_language",
  "source_code"
]
}
```

Save

Cancel

Step 6: The custom function **convert_code** has been successfully created and is now ready to use.

Functions

+ Add function

fx

convert_code

...

Step 7: In the query box, enter your prompt and press **Enter** to test the function.

Prompt: convert the below code into python"public class PalindromeNumber
{

```
    public static void main(String[] args) {
```

```
        int number = 121;
```

```
        int original = number;
```

```
        int reversed = 0;
```

```
        while (number != 0) {
```

```
            int digit = number % 10;
```

```
            reversed = reversed * 10 + digit;
```

```
            number = number / 10;
```

```
        }
```

```
        if (original == reversed) {
```

```
            System.out.println("Palindrome Number");
```

```
        } else {
```

```
            System.out.println("Not a Palindrome Number");
```

```
        }
```

```
    }
```

```
}"
```

Clear chat Logs Thread files JSON response 0 tokens



Start chatting

Test your Assistant by sending queries below. Then adjust your Assistant setup to improve the Assistant's responses.

Type user query here. (Shift + Enter for new line)

Step 8: Once you type your prompt and press **Enter**, you will see the function call generated as shown below. Now, click the **Play** button to execute it.

Clear chat Logs Thread files JSON response 0 tokens

```
System.out.println("Not a Palindrome String");
}
```



```
convert_code({"source_language":"java","target_language":"python","source_code":"public class PalindromeStringManual {\n public\n static void main(String[] args) {\n String str = \"madam\";\n boolean isPalindrome = true;\n for (int i = 0; i < str.length() / 2; i++) {\n if (str.charAt(i) != str.charAt(str.length() - 1 - i)) {\n isPalindrome = false;\n break;\n }\n }\n if (isPalindrome) {\n System.out.println(\"Palindrome String\");\n } else {\n System.out.println(\"Not a Palindrome String\");\n }\n }\n}","conversion_type":"idiomatic","include_comments":true})
```

Submit output e.g. {success: true}



Step 9: After clicking the **Play** button, the converted output is displayed as shown below.

Submit output e.g. {success: true}



Here is the converted Python code from the provided Java code:

```
def is_palindrome(s):
    # Assume the string is a palindrome initially
    is_palindrome = True

    # Check characters from the start and the end moving towards the center
    for i in range(len(s) // 2):
        if s[i] != s[len(s) - 1 - i]:
            is_palindrome = False
            break
```